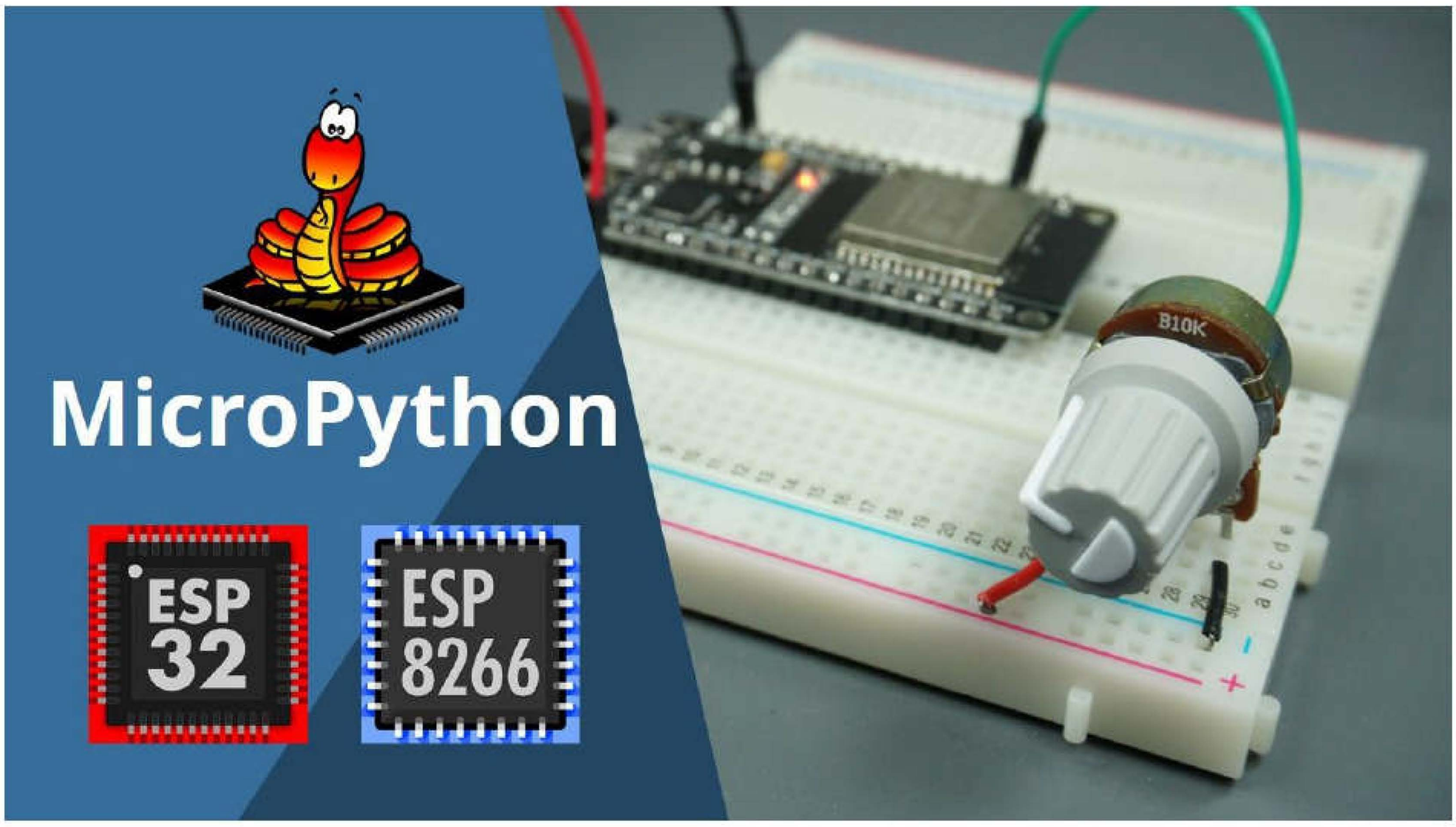
Analog Inputs



In this Unit, you'll learn how to read an analog input. This is useful to read values from variable resistors like potentiometers, or analog sensors.

Project Overview

In the previous Unit you've learned how to detect if a GPIO is on (3.3V) or off (0V). Using ADC (analog to digital converter) GPIOs allow you to read voltages between 0V and 3.3V.

The voltage measured is then assigned to a value between 0 and 4095 (in case of the ESP32), or between 0 and 1023 (in case of the ESP8266) in which 0V corresponds to 0, and 3.3V corresponds to 4095 (or 1023 for the ESP8266). Any voltage between 0V and 3.3V is given the corresponding value in between.

In this section, we'll build a simple example that reads the analog values from a potentiometer.

Analog reading with ESP8266

ESP8266 only has one analog pin called ADC 0. The ESP8266 analog pin has 10-bit resolution. It reads the voltage from 0 to 3.3V and then, assigns a value between 0 and 1023.

> **Note:** some versions of the ESP8266 only read a maximum of 1V on the ADC pin. Make sure you don't exceed the maximum recommended voltage for your board.

Analog reading with ESP32

There are several pins on the ESP32 that can act as analog pins - these are called ADC pins. All the following GPIOs can act as ADC pins: 0, 2, 4, 12, 13, 14, 15, 25, 26, 27, 32, 33, 34, 35, 36, and 39.

ESP32 ADC pins have 12-bit resolution by default. These pins read voltage between 0 and 3.3V and then return a value between 0 and 4095. The resolution can be changed on the code. For example, you may want to have just 10-bit resolution to get a value between 0 and 1023.

The following table shows some differences between analog reading on the ESP8266 and the ESP32 board.

	ESP8266	**ESP32**
Analog pins	ADC 0	GPIOs: 0, 2, 4, 12, 13, 14, 15, 25, 26, 27, 32, 33, 34, 35, 36, and 39.
Resolution	10-bit (0-1023)	12-bit (0-4095)
Change resolution in the code	No	Yes

Analog reading works differently in ESP32 and ESP8266. There is a different schematic and a different script for each board.

Schematic

Before proceeding with this Unit, you need to connect a potentiometer to your ESP32 or ESP8266 board.

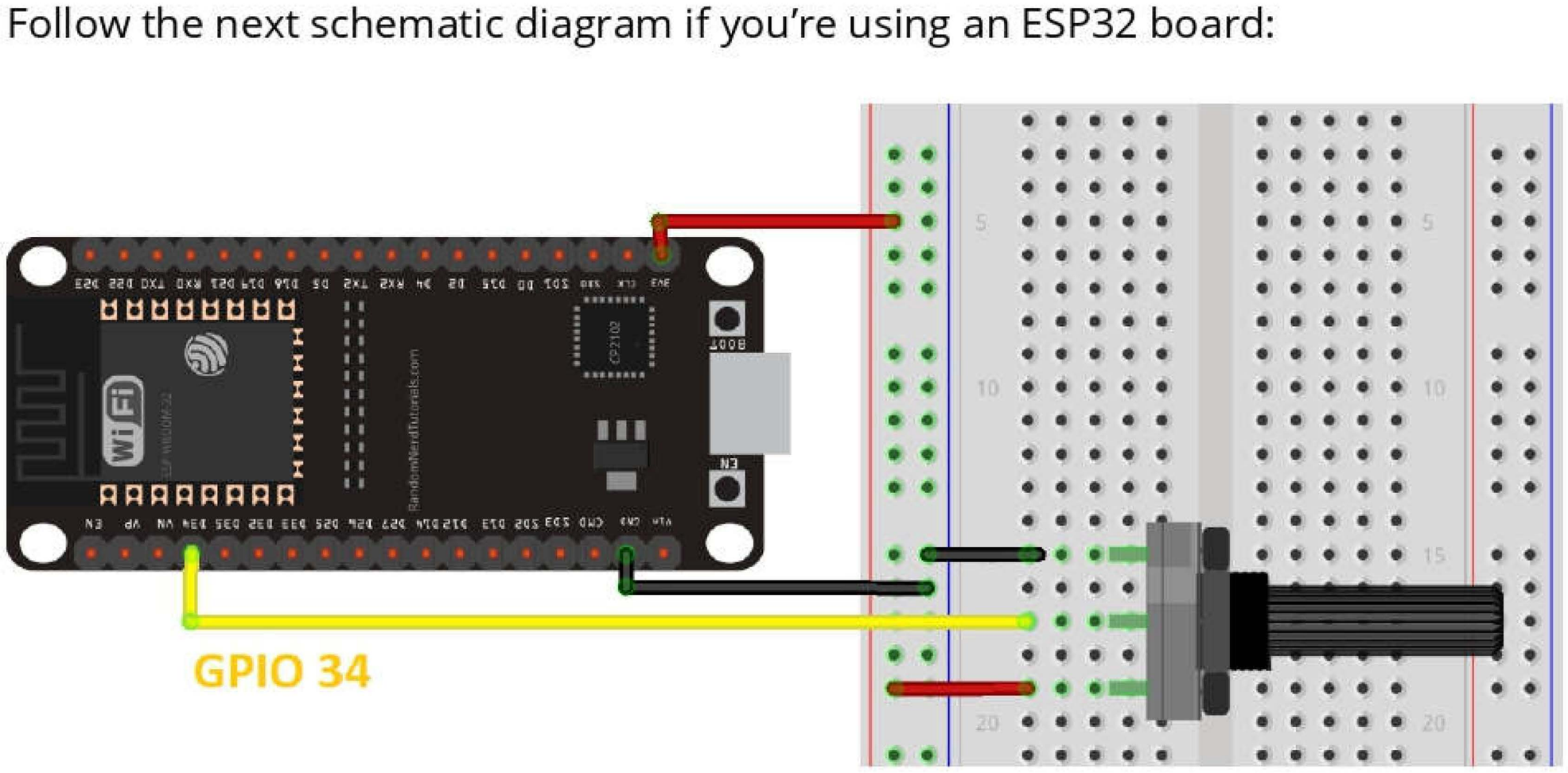
Parts required

Here's a list of parts you need to build the circuit:

- ESP32 or ESP8266
- Potentiometer
- Breadboard
- Jumper wires

Schematic – ESP32

Follow the next schematic diagram if you're using an ESP32 board:

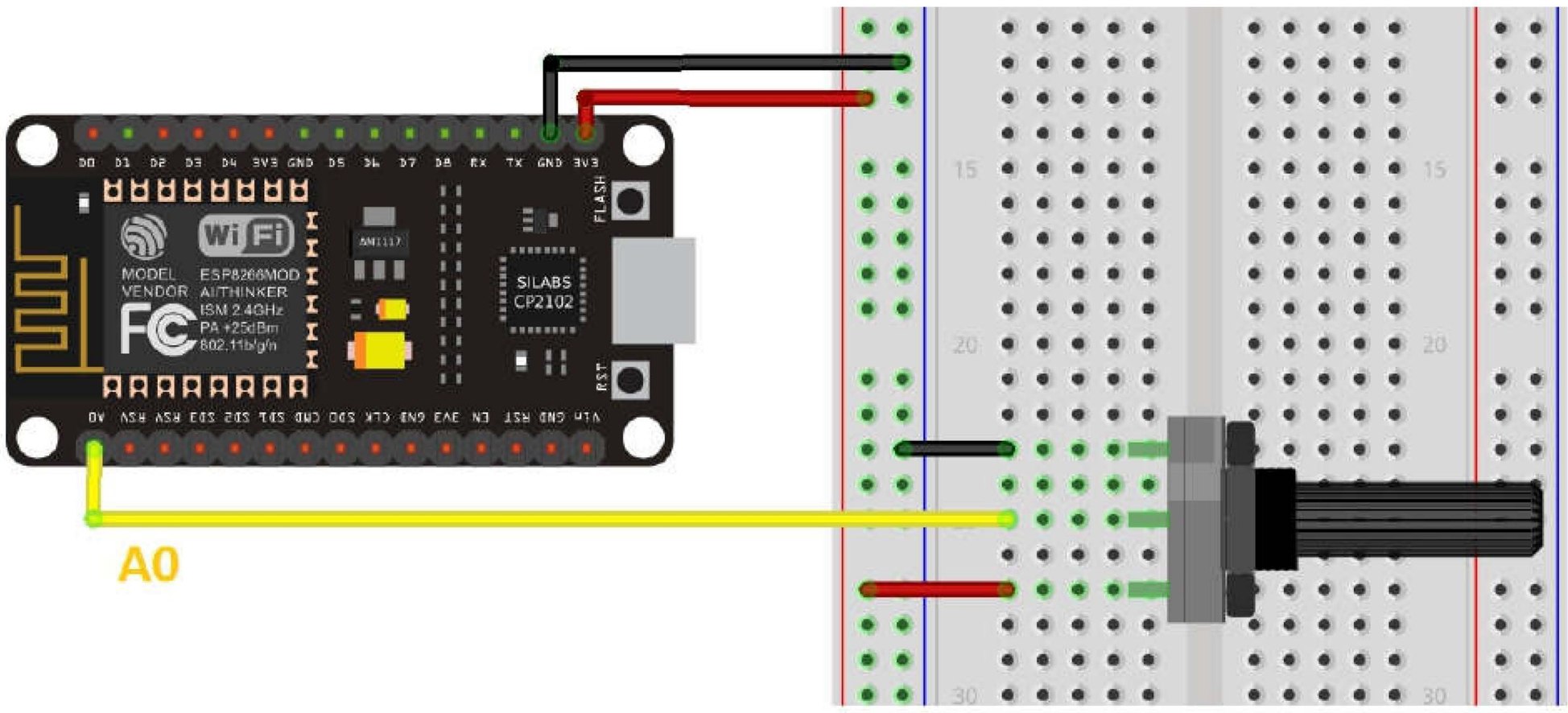


This schematic uses the ESP32 DEVKIT DOIT board version with 36 GPIOs. Before assembling the circuit double-check the pinout for the board you're using.

Note: in this example we're using GPIO 34 to read analog values from the potentiometer, but you can choose any other GPIO that supports ADC.

Schematic – ESP8266

Follow the next schematic diagram if you're using an ESP8266 board:



Script

There are a few differences when it comes to analog reading in ESP32 and ESP8266 regarding the code. You should write a slightly different script depending on the board you're using. Make sure you follow the code for your specific board.

Script - ESP32

The following script for the ESP32 reads analog values from GPIO 34.

```
from machine import Pin, ADC
from time import sleep

pot = ADC(Pin(34))
pot.atten(ADC.ATTN_11DB)  #Full range: 3.3v

while True:
  pot_value = pot.read()
  print(pot_value)
  sleep(0.1)
```

SOURCE CODE
https://github.com/RuiSantosdotme/ESP-MicroPython/blob/master/code/GPIOs/Analog_Read_Pot/read_pot_esp32.py

How the code works

To read analog inputs, import the `ADC` class in addition to the `Pin` class from the `machine` module. We also import the `sleep` method.

```
from machine import Pin, ADC
from time import sleep
```

Then, create an `ADC` object called `pot` on GPIO 34.

```
pot = ADC(Pin(34))
```

The following line defines that we want to be able to read voltage in full range.

```
pot.atten(ADC.ATTN_11DB)
```

This means we want to read voltage from 0 to 3.3V. This corresponds to setting the attenuation ratio of 11db. For that, we use the `atten()` method and pass as argument: `ADC.ATTN_11DB`

The `atten()` method can take the following arguments:

- **ADC.ATTN_0DB** — the full range voltage: 1.2V
- **ADC.ATTN_2_5DB** — the full range voltage: 1.5V
- **ADC.ATTN_6DB** — the full range voltage: 2.0V
- **ADC.ATTN_11DB** — the full range voltage: 3.3V

In the `while` loop, read the `pot` value and save it in the `pot_value` variable. To read the value from the pot, simply use the `read()` method on the `pot` object.

```
pot_value = pot.read()
```

Then, print the pot value.

```
print(pot_value)
```

At the end, add a delay of 100 ms.

```
sleep(0.1)
```

When you rotate the potentiometer, you get values from 0 to 4095 - that's because the ADC pins have a 12-bit resolution by default. You may want to get values in

other ranges. You can change the resolution using the `width()` method as follows:

```
ADC.width(bit)
```

The `bit` argument can be one of the following parameters:

- **ADC.WIDTH_9BIT**: range 0 to 511
- **ADC.WIDTH_10BIT**: range 0 to 1023
- **ADC.WIDTH_11BIT**: range 0 to 2047
- **ADC.WIDTH_12BIT**: range 0 to 4095

For example:

```
ADC.width(ADC.WIDTH_12BIT)
```

In summary:

- To read an analog value you need to import the `ADC` class;
- To create an ADC object simply use `ADC(Pin(GPIO))`, in which GPIO is the number of the GPIO you want to read the analog values;
- To read the analog value, simply use the `read()` method on the `ADC` object.

Script – ESP8266

The following script for the ESP8266 reads analog values from ADC 0 pin.

```
from machine import Pin, ADC
from time import sleep

pot = ADC(0)

while True:
  pot_value = pot.read()
  print(pot_value)
  sleep(0.1)
```

SOURCE CODE

https://github.com/RuiSantosdotme/ESP-MicroPython/blob/master/code/GPIOs/Analog_Read_Pot/read_pot_esp8266.py

How the code works

To read analog inputs, import the `ADC` class in addition to the `Pin` class from the `machine` module. We also import the `sleep` method.

```
from machine import Pin, ADC
from time import sleep
```

Then, create an `ADC` object called `pot` on ADC 0 pin.

```
pot = ADC(0)
```

Note: ADC0 (A0) is the only pin on the ESP8266 that supports analog reading.

In the loop, read the pot value and save it in the `pot_value` variable. To read the value from the `pot`, use the `read()` method on the `pot` object.

```
pot_value = pot.read()
```

Then, print the `pot_value`.

```
print(pot_value)
```

At the end, add a delay of 100 ms.

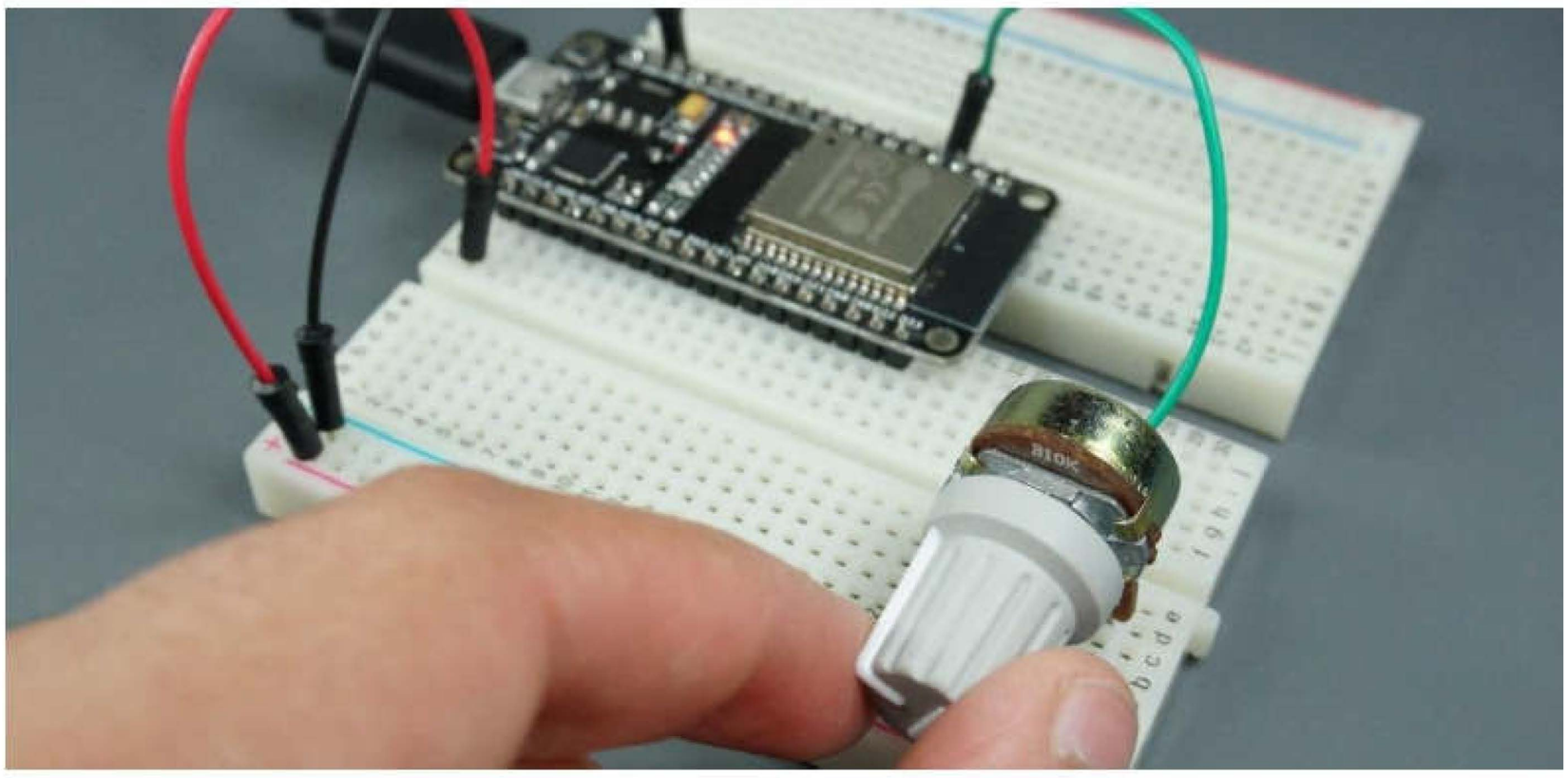
```
sleep(0.1)
```

In summary:

- To read an analog value you use the `ADC` class;
- To create an ADC object simply call `ADC(0)`. The ESP8266 only supports ADC reading on ADC 0 pin.
- To read the analog value, use the `read()` method on the `ADC` object.

Demonstration

After uploading the code to your board, rotate the potentiometer.



In the uPyCraft Shell, if you're using an ESP32 you should get readings between 0 and 4095 — or readings between 0 and 1023 with an ESP8266.